BookLeaf Author Support Portal: Technical Write-up

Approach and priorities

I built the full support workflow first and kept the UI deliberately plain. The ticket flow and the AI integration come first because they carry the most weight in the brief. Authors view their books and royalties, raise tickets and follow replies. Admins get a prioritised queue and can assign tickets, change status, override AI classifications, add internal notes and send replies.

The stack is React + TypeScript (Vercel), FastAPI + SQLAlchemy (Render) and PostgreSQL (Neon). The backend is a modular monolith: thin routes, business rules in services, and all AI code in one `ai/` package behind a provider interface, so changing the model is a config change. Auth uses a JWT in an httpOnly cookie with CSRF protection, and every route checks role and ownership server-side. Secrets stay in environment variables, and internal notes live in their own table so they never reach authors or prompts.

AI integration

AI does two jobs: classify each new ticket (category and priority in one call) and draft a reply when an admin first opens it.

The key design decision is that **the model writes the wording, while code supplies the facts**. Policies like Royalty totals, the ₹1,000 threshold, the next payout date and overdue status are computed in SQL and Python and passed in as a short facts block. The prompt allows only those numbers and tells the model to list missing information (e.g. bank details) instead of guessing.

Each draft prompt contains the tone guidelines, the top two knowledge-base chunks, one or two sample responses, the facts block. PostgreSQL full-text search retrieves the policy chunks, so no vector database is needed. Author text is treated as data, as a prompt-injection guard.

For cost control, drafts are generated on demand and cached, context and output are capped, and each call is logged with tokens and latency. If classification fails, keyword rules still sort the ticket. If drafting fails,the editor loads a generic holding reply instead, which the admin can edit or replace before sending. I chose Gemini 3.8 Flash lite because it is free, stable and supports schema-enforced JSON.

Trade-offs

I chose a monolith over microservices, and SSE over WebSockets since updates only flow server to browser. The event broker is single-process, so it needs shared delivery to scale. “Overdue” royalty status is an approximation, as the data has no per-quarter sales.

Production path

I would track classification accuracy, AI cost, failures and override rate, and move to a paid model tier because free-tier prompts may be used by the provider and author data is private. I would add Redis or LISTEN/NOTIFY for events, refresh tokens, email notifications, SLA alerts and a per-quarter royalty ledger.